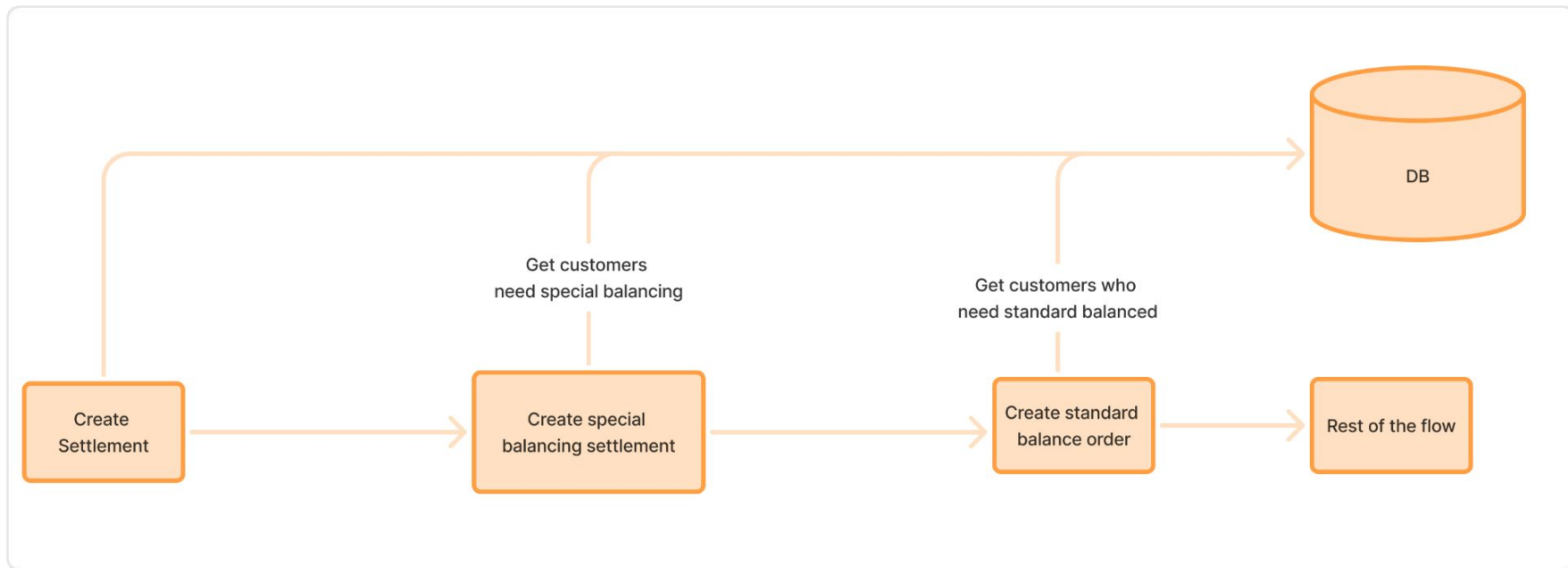
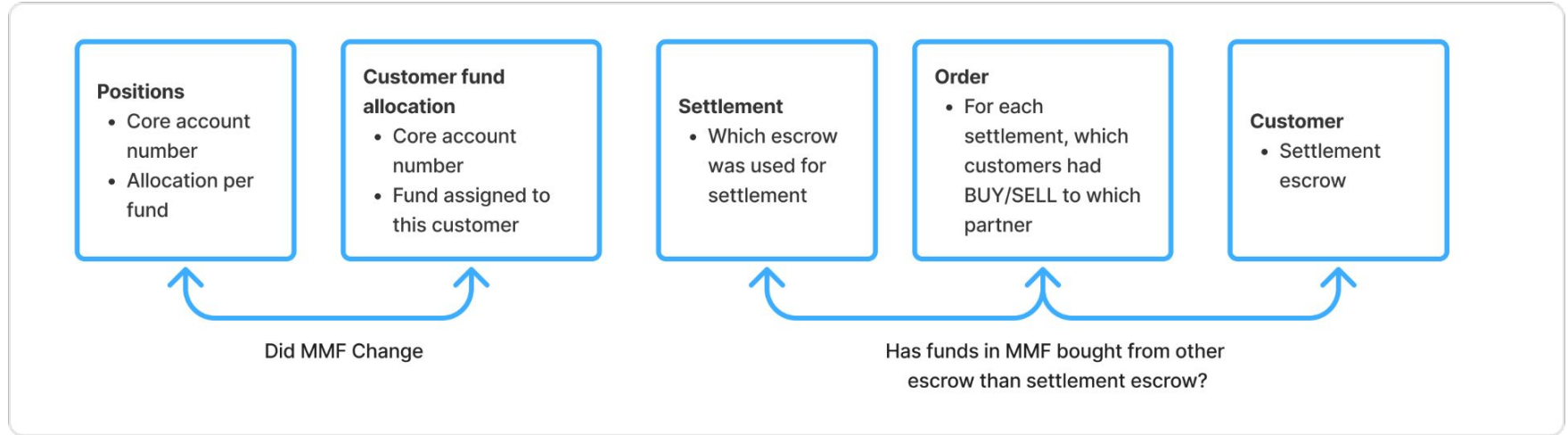


Scaling Data Processing in Kotlin without Melting Your Memory.

Backstory



Entities in DB



ProcessEligibleCustomers

MMF: Amundi

Matched: 10% of total data

Duration: 30mins



ProcessEligibleCustomers

MMF: DWS

Matched: 40% of total data

Duration: 2h



ProcessEligibleCustomers

MMF: Blackrock

Matched: 50% of total data

Duration: 3h

- **A single mega query to read all data. Taking between 20mins to 3h depending on filters used**
- **Fetch with JOOQ streams**
- **Chunks with sequences**

- **Completed intra fund orders before MMF Settlement runs**
- **Don't need to add co-routines**
- **Service memory efficient disregard of amount of data loaded**

Eager List Allocations vs. "Millions of Short-Lived Objects"

Eager List Allocations vs. "Millions of Short-Lived Objects"

Imagine you are processing **100,000 eligible customers**. With transforming, filtering and returning first 10 valid ones. The JVM evaluates everything **eagerly**.

STEP 1: .filter

Process All customers

Runs on all **100,000** customers. Creates a new List in the Heap.

STEP 2: .map

Transform Everything

Runs on the filtered list. Creates **another** new List object.

STEP 3: .take(10)

The Final Output

Only keep the first **10** results.

The Result: 200,000 "Short-Lived Objects"

JVM allocated memory for 200k objects just to use 10.

These intermediate lists become garbage almost immediately after creation.



Loads all matched data at once

Improve application memory with with Sequences

How: Going Lazy with Sequences

Sequences process one item at a time through the entire pipeline.

Step 1

Item 1

Goes through filter **fails** and is immediately **discarded**.

Step 2

Item 2

Filter **passes**, maps, and is **saved** successfully.

Final Step

Termination

At the 10th valid item, **the whole process stops** instantly.

Because no intermediate collections are created, you **drastically reduce** the number of temporary objects hitting the JVM Heap.



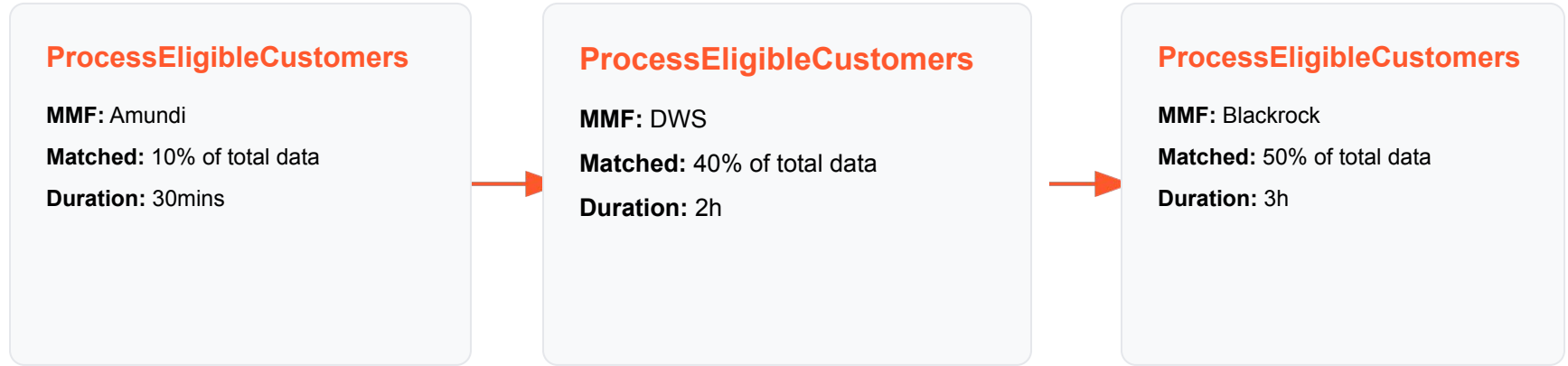
Single-Threaded Execution

While memory is saved, it's still running **sequentially** on a single thread.



Latency Sensitivity

It's safe, but not necessarily fast if individual steps like **DB or API calls** are slow.



Total processing time = 5h30

Lazy fetch from DB with Jooq

List

fetch

Results in a Java List (eager)

Cursor

fetchLazy

Internal reference to `ResultSet`
Requires `fetchSize`

Stream

Uses Java Stream API
Holds internal reference to
`ResultSet`

```
// query implementation  
.fetch()
```

```
// query implementation  
.fetchSize(Int)  
.fetchLazy()
```

```
// query implementation  
.fetchSize(Int)  
.fetchStream()  
.asSequence()
```


The Mechanism

FetchStream + FetchSize (DB)

Execute once the query

Fetch the first 10k in a single round trip

Cursor holds pointer to ResultSet

Sequence + chunkSize (application)

Process items in batches

Why it works

- Prevents "Out of Memory" errors by not loading the entire result set.
- Balances database server cursor resources with application processing speed.
- Enables smooth, reactive data pipelines for high-volume processing.

Phase 1: Query Plan Execution (ONCE)

```
SELECT * FROM customers WHERE eligible = true ORDER BY id
```

Matches 20,000 rows

Returns first 10,000

Phase 2: Server-Side Cursor (ACTIVE)

Holds the result set and maintains the read state until the stream is fully consumed.

Read Position:



^ Current read position

Row Index: [1] [2] [3] ... [9999] [10000] [10001] ... [20000]

TCP/Network ▼

JDBC DRIVER

Fetch Buffer (fetchSize = 10,000)

Holds up to 10,000 rows in memory

Current: [1] [2] [3] ... [9998] [9999] [10000]

ResultSet.next() ▼

JOOQ STREAM / KOTLIN SEQUENCE

`.fetchStream().asSequence()` → Lazy iterator over `ResultSet`

`.chunked(chunkSize = 10,000)` ▼

APPLICATION PROCESSING

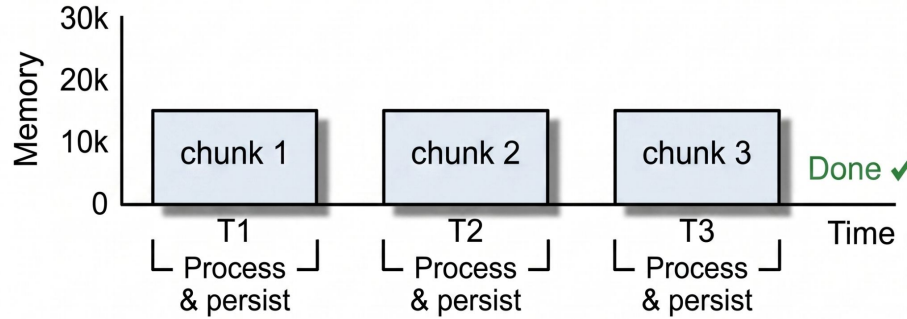
Chunk 1: [1..10,000] → `persistOrderBatch()` → ✓ Done

Chunk 2: [10,001..20,000] → (waiting...)

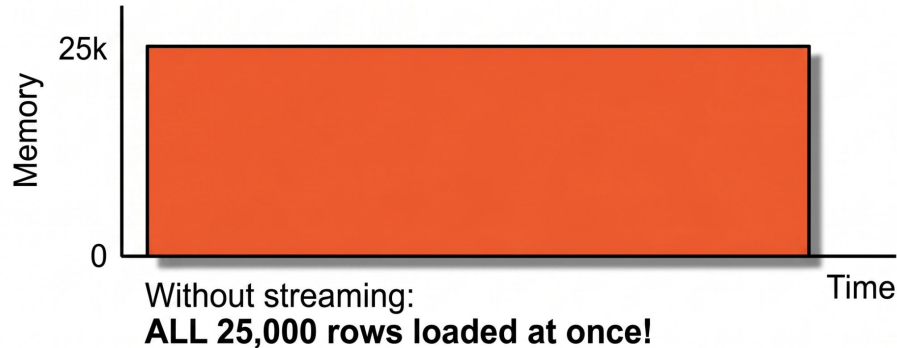
Chunk 3: [20,001..25,000] → (not yet)

Memory Usage Analysis

Memory Usage with efficient Chunking (Low Stable Peak)

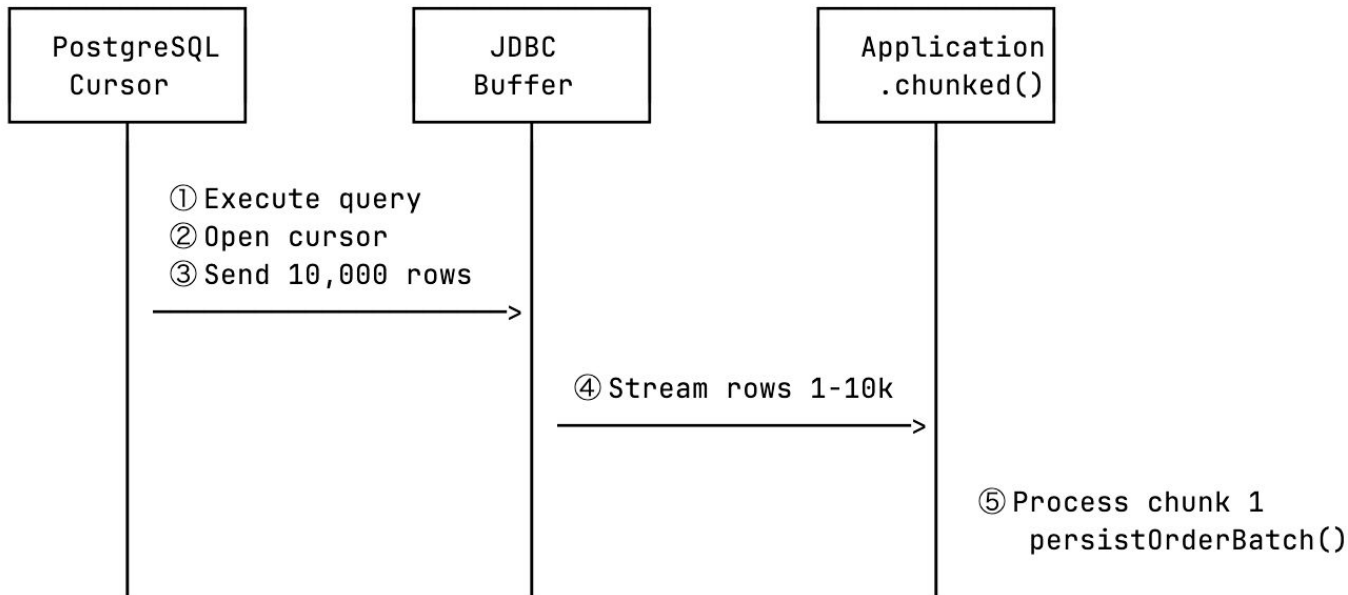


Memory Usage without streaming (High Peak Load)



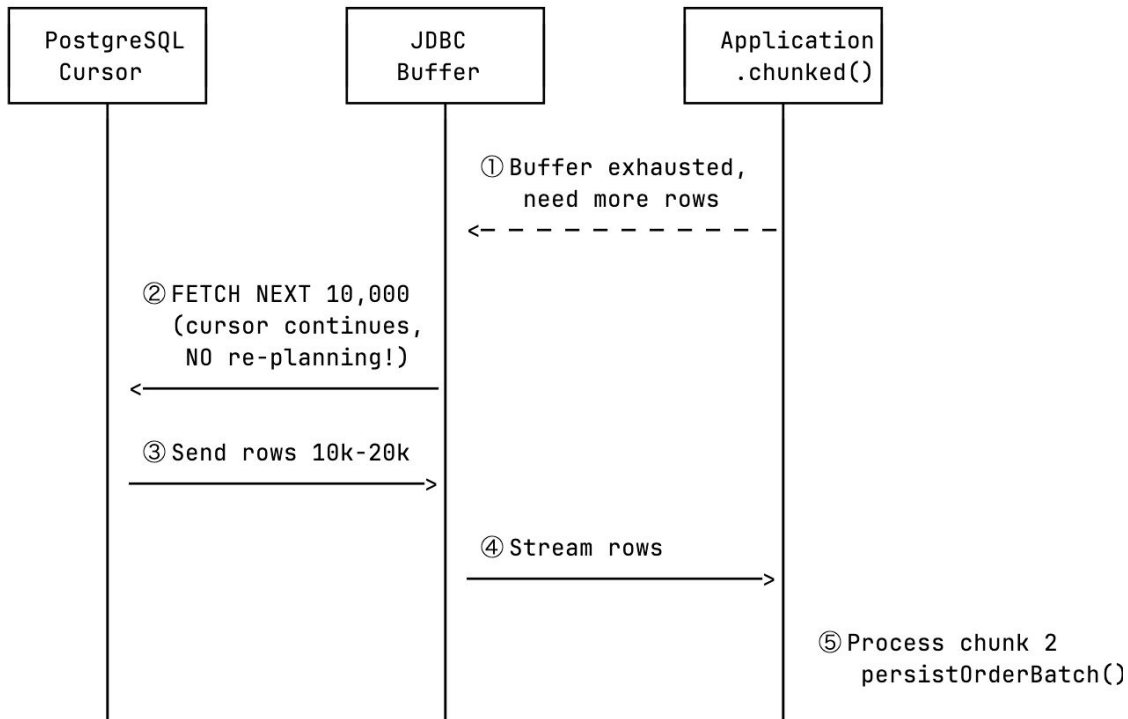
What happens

FIRST FETCH (rows 1 - 10,000)



What happens

SECOND FETCH (rows 10,001 - 20,000)



Unleashing Speed with Flow



Asynchronous & Sequential: Light threads for asynchronous programming in a sequential manner.



Simplify Operations: Effortlessly handle long-running operations without complexity.



Non-blocking: Handle time-consuming tasks without freezing the underlying thread.



Resource Efficient: Significantly more lightweight than traditional OS threads.



Structured Concurrency: Manage lifecycles to avoid leaks and inconsistent states.

**Asynchronous & Advanced abstraction:**

An advanced abstraction of coroutines for modern concurrent programming.

**Non-blocking Operations:**

Suspends during DB or network I/O, freeing the underlying thread for other tasks until data is ready.

**Cold Streams:**

Code inside a flow { ... } builder doesn't actually run until someone calls `.collect()` on it.

Sequential flow

```
flowOf("Amundi", "DWS", "Blackrock")  
.collect { mmf -> processEligibleCustomers (mmf) }
```

ProcessEligibleCustomers

MMF: Amundi

Matched: 10% of total data

Duration: 30mins



ProcessEligibleCustomers

MMF: DWS

Matched: 40% of total data

Duration: 2h



ProcessEligibleCustomers

MMF: Blackrock

Matched: 50% of total data

Duration: 3h

Total processing time = 5h30

Asynchronous Concurrency (flatMapMerge)

```
val mmfs = listOf("Amundi", "DWS", "Blackrock")

mmfs.asFlow()
  .flatMapMerge(concurrency = 3) { mmf ->
    // Strictly limits DB reads to 3 at a time
    flow {
      emit(db.readLargeEligibleDataset(mmf)) Returns a sequence
    }
  }
  .map { customer -> processLightweightAppLogic(customer) }
  .collect { result -> db.writeResult(result) }
```



The application opens **three separate database connections/cursors at the same time**.

Asynchronous Concurrency (flatMapMerge)



This way has to wait until the entire database read is completely finished before it passes the massive chunk downstream.

Flatten the stream from DB with flow

```
fun readLargeEligibleDataset(mmf: String): Flow<Customer> = flow {  
    // 1. Setup the lazy query configuration  
    val lazyQuery = dbContext.selectFrom(CUSTOMERS)  
        .where(CUSTOMERS.MMF.eq(mmf))  
        .fetchLazy()  
        .fetchSize(100) // Fetch from network in batches of 100  
  
    try {  
        // 2. Stream rows continuously out of the open cursor  
        for (row in lazyQuery) {  
            emit(row.toCustomerDomainModel()) // Push single customer down the stream  
        }  
    } finally {  
        lazyQuery.close() // Crucial: Always close the cursor when the flow completes  
    }  
}
```

Processing pipeline

```
val mmfs = listOf("Amundi", "DWS", "Blackrock")

mmfs.asFlow()

// 1. flatMapMerge opens 3 concurrent DB streams at the same time
.flatMapMerge(concurrency = 3) { mmf ->
db.readLargeEligibleDataset(mmf) // Returns Flow<Customer>
}

// 2. Lightweight processing happens on individual customers as they stream in
.map { customer ->
processLightweightAppLogic(customer)
}

// 3. Write results back to the database as they finish processing
.collect { result ->
db.writeResult(result)
}
```

Asynchronous Concurrency (flatMapMerge)

Total execution time is determined by the slowest parallel task

Amundi

Matched: 10% data

Duration: 30 mins

DWS

Matched: 40% data

Duration: 2h

Blackrock (Critical Path)

Matched: 50% data

Duration: 3h

Total duration of the longest job: 3h



Database

Database Connection Pool Starvation

```
flatMapMerge(concurrency = 20)
```



Pool Limit = 10



The Catch:

Other parts of the service will be **waiting** for connections to free up, causing system-wide latency or failures.

When to use which

When to use which

List

Small Data

Best for eager processing of collections that fit comfortably in memory.

Sequence

Large Data

Single-threaded data processing.

Efficient for large datasets via lazy evaluation.

Flow

Async Data

Reactive data streams.

Handles asynchronous and cold streams effectively.



THANK
YOU!

